# 배포 정리

## 환경 변수

각 서비스마다 Jenkins Credential로 관리

1. application.yml

- eureka-server

```
spring:
  application:
    name: eureka-server

server:
  port: 8761

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false

management:
  endpoints:
    web:
      exposure:
        include: "*"
```

- api-gateway

```
server:
  port: 8080

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true
```

```
spring:
  main:
    web-application-type: reactive
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://MEMBER-SERVICE
          predicates:
            - Path=/api/v1/auth/**

        - id: member-service
          uri: lb://MEMBER-SERVICE
          predicates:
            - Path=/api/v1/members/**

        - id: payment-service
          uri: lb://PAYMENT-SERVICE
          predicates:
            - Path=/api/v1/payments/**, /internal/**

        - id: notification-service
          uri: lb://NOTIFICATION-SERVICE
          predicates:
            - Path=/api/v1/notifications/**

        - id: vehicle-service
          uri: lb://VEHICLE-SERVICE
          predicates:
            - Path=/api/v1/vehicles/**

        - id: remote-service
          uri: lb://REMOTE-SERVICE
          predicates:
            - Path=/api/v1/remote/**
```

```
      globalcors:
        cors-configurations:
          '[/**]':
            allowedOrigins:
              - "http://localhost:3000"
              - "http://localhost:5173"
              - "https://j12a707.p.ssafy.io"
            allowedMethods:
              - GET
              - POST
              - PUT
              - DELETE
              - OPTIONS
            allowedHeaders: "*"

logging:
  level:
    org.springframework.cloud.gateway: DEBUG
```

- member-service

```
server:
  port: 8081

spring:
  application:
    name: member-service
  datasource:
    url: jdbc:mysql://mysql:3306/vanish_user_db?useSSL=false&serverTimezor
    username: root
    password: ssafy
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  security:
```

```
    user:
      name: user
      password: password
  mail:
    host: smtp.gmail.com
    port: 587
    username: dldltjrgusrhrl@gmail.com
    password: lmfppjkbtuyukmjm
    protocol: smtp
    properties:
      mail:
        smtp:
          auth: true
          starttls:
            enable: true

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true

jwt:
  private-key-path: private_key.pem
  public-key-path: public_key.pem
  access-token-validity: 60  # 분 단위 (1시간)
  refresh-token-validity: 1440  # 분 단위 (1일)
```

- notification-service

```
server:
  port: 8083  # 알림 서비스 포트

spring:
  application:
```

```
    name: notification-service
  rabbitmq:
    host: rabbitmq
    port: 5672
    username: guest
    password: guest
  datasource:
    url: jdbc:mysql://mysql:3306/vanish_notification_db?useSSL=false&serverT
    username: root
    password: ssafy
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
    register-with-eureka: true
    fetch-registry: true
  instance:
    prefer-ip-address: true

jwt:
  public-key-path: public_key.pem
```

- payment-service

```
server:
  port: 8082

spring:
  application:
    name: payment-service
  datasource:
    url: jdbc:mysql://mysql:3306/vanish_payment_db?useSSL=false&serverTim
```

```
    username: root
    password: ssafy
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  rabbitmq:
    host: rabbitmq
    port: 5672
    username: guest
    password: guest

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
    instance:
      prefer-ip-address: true

jwt:
  public-key-path: public_key.pem

imp:
  api: 0705887027267851
  secretkey: wfwUsY2n1dUZLYIHyvbCG9O5JIYbqrTIOaqxbmmCmbmxJHgjBK

rabbit:
  vehicle:
    exchange: ros_exchange
    routing-key: vehicle.departure
```

- remote-service

```
server:
  port: 8085  # 원격 제어 서비스 포트
```

```
spring:
  application:
    name: remote-service
  datasource:
    url: jdbc:mysql://mysql:3306/vanish_remote_db?serverTimezone=UTC&cha
    username: root
    password: ssafy
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    database-platform: org.hibernate.dialect.MySQL8Dialect
  rabbitmq:
    host: rabbitmq
    port: 5672
    username: guest
    password: guest
    listener:
      simple:
        auto-startup: true
    ros:
      routingKey: remote.parking
    destination:
      exchange: remote_exchange
      routingKey: remote.destination
      reply:
        routingKey: remote.destination.reply
    roam:
      routingKey: remote.roam

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
    register-with-eureka: true
    fetch-registry: true
```

```
  instance:
    prefer-ip-address: true

jwt:
  public-key-path: public_key.pem
```

- vehicle-service

```
server:
  port: 8084  # 차량 서비스 포트

spring:
  data:
    redis:
      host: redis
      port: 6379
  application:
    name: vehicle-service
  datasource:
    url: jdbc:mysql://mysql:3306/vanish_vehicle_db?serverTimezone=UTC&cha
    username: root
    password: ssafy
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    database-platform: org.hibernate.dialect.MySQL8Dialect
  rabbitmq:
    host: rabbitmq
    port: 5672
    username: guest
    password: guest
    listener:
      simple:
        auto-startup: true

eureka:
```

```
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
    register-with-eureka: true
    fetch-registry: true
  instance:
    prefer-ip-address: true
```

## 도커 컨테이너 정리

- Jenkins

```
docker pull jenkins/jenkins:lts-jdk17

# pull 받은 이미지 확인
docker images ls

docker volume create jenkins-volume

docker run -d \
  -p 8081:8080 \
  -p 50000:50000 \
  -v jenkins-volume:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add $(stat -c '%g' /var/run/docker.sock) \
  --name jenkins \
  jenkins/jenkins:lts-jdk17
```

- 각 비즈니스 서비스(ex vehicle-service)

```
# Dockerfile

# 경량 JDK 17 이미지 사용
FROM openjdk:17-jdk-slim

# 임시 디렉토리 생성 (Spring Boot가 사용하는 곳)
VOLUME /tmp
```

```
# 빌드 결과로 생성된 JAR 파일 경로 (gradlew 빌드 기준)
ARG JAR_FILE=build/libs/vehicle-service-0.0.1-SNAPSHOT.jar

# 해당 JAR 파일을 이미지에 복사
COPY ${JAR_FILE} app.jar

# 애플리케이션 실
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Jenkinsfile
pipeline {
    agent any

    environment {
        REGISTRY = 'taemin0323'
        CREDENTIALS_ID = 'Docker-Hub'
        SERVICE_NAME = 'vehicle-service'
        SERVICE_PORT = '8084'
        NETWORK_NAME = 'vanish-net'
        TARGET_DIR = 'BE/vehicle-service'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                echo 'Git 소스코드 체크아웃 완료'
                script {
                    // Shallow clone 상태라면 전체 이력을 가져오도록 처리
                    def isShallow = sh(
                            script: 'git rev-parse --is-shallow-repository',
                            returnStdout: true
                    ).trim()

                    if (isShallow == 'true') {
                        echo "🔄 shallow clone 감지 → git fetch --unshallow 수행"
                        sh 'git fetch --unshallow'
                    } else {
```

```
                echo "✅ 이미 full clone 상태입니다"
            }

            withCredentials([string(credentialsId: 'gitlab-jenkins-token', variab
                sh '''
                    git remote set-url origin https://git:$GIT_TOKEN@lab.ssafy.co
                    git fetch origin develop-BE
                '''
            }
        }
    }
}

stage('Check Directory Changes') {
    steps {
        script {
            def diffFiles = sh(script: "git diff --name-only HEAD~1 HEAD", retu
            echo "변경된 파일 목록:\n${diffFiles}"

            def hasChanges = diffFiles.tokenize('\n').any { file -> file.startsWit
            def isManual = currentBuild.rawBuild.getCause(hudson.model.Cau

            if (!hasChanges && !isManual) {
                echo "${TARGET_DIR} 변경 없음. 자동 빌드 중단."
                currentBuild.result = 'NOT_BUILT'
                error("No relevant changes. Skipping build.")
            } else if (hasChanges) {
                echo "변경 감지됨. 빌드 계속 진행."
            } else {
                echo "수동 실행으로 계속 진행."
            }
        }
    }
}

stage('Copy application.yml') {
    steps {
        withCredentials([file(credentialsId: 'vehicle-service-application', vari
```

```
                script {
                    dir("${TARGET_DIR}/src/main/resources") {
                        sh 'rm -f application.yml || true'
                        sh 'cp $application application.yml'
                        sh 'chmod 644 application.yml'
                    }
                }
            }
        }

        stage('Copy secretFiles') {
            steps {
                withCredentials([file(credentialsId: 'vehicle-public-key', variable: 'PU
                    script {
                        dir("${TARGET_DIR}/src/main/resources") {
                            sh 'cp $PUBLIC_KEY public_key.pem'
                            sh 'chmod 644 public_key.pem'
                        }
                    }
                }
            }
        }

        stage('Build with Gradle') {
            steps {
                dir("${TARGET_DIR}") {
                    sh 'chmod +x ./gradlew'
                    sh './gradlew clean build -x test'
                }
            }
        }

        stage('Docker Build & Push') {
            steps {
                script {
                    def commitHash = sh(script: "git rev-parse --short HEAD", returnS
                    env.IMAGE_TAG = commitHash
```

```
                dir("${TARGET_DIR}") {
                    docker.withRegistry('https://index.docker.io/v1/', CREDENTIALS
                        def image = docker.build("${REGISTRY}/${SERVICE_NAME}:
                        image.push("${commitHash}")
                        image.push("latest")
                    }
                }
            }
        }

        stage('Docker Deploy') {
            steps {
                script {
                    sh "docker stop ${SERVICE_NAME} || true"
                    sh "docker rm ${SERVICE_NAME} || true"
                    sh "docker pull ${REGISTRY}/${SERVICE_NAME}:${IMAGE_TAG}"
                    sh "docker run -d --name ${SERVICE_NAME} --network ${NETW
                    echo "${SERVICE_NAME} 배포 완료 (버전: ${IMAGE_TAG})"
                }
            }
        }
    }
}
```

- Mysql

```
docker pull mysql:8.0

docker run -d \
  --name mysql \
  --network vanish-net \
  -e MYSQL_ROOT_PASSWORD=ssafy \
  -p 3306:3306 \
  -v mysql_data:/var/lib/mysql \
  mysql:8.2.0 \
  --character-set-server=utf8mb4 \
```

```
  --collation-server=utf8mb4_unicode_ci

  # mysql 컨테이너 내부 접
  docker exec -it mysql bash
```

- RabbitMQ

```
docker pull rabbitmq

docker run -d \
  --name rabbitmq \
  --network vanish-net \
  -p 3001:5672 \ # 메시징 포트 (서비스용)
  -p 3002:15672 \ # 웹 UI 포트 (관리용)
  rabbitmq

docker exec rabbitmq rabbitmq-plugins enable rabbitmq_management
```

- FRONTEND

```
# Dockerfile

# 1단계: React 빌드
FROM node:22 AS builder
WORKDIR /app

# 빌드 인자를 사용해 VITE_MAPBOX_ACCESS_TOKEN 변수를 전달
ARG VITE_MAPBOX_ACCESS_TOKEN
ENV VITE_MAPBOX_ACCESS_TOKEN=${VITE_MAPBOX_ACCESS_TOKEN}

ARG VITE_IMP_CODE
ENV VITE_IMP_CODE=${VITE_IMP_CODE}

COPY . .
RUN npm install && npm run build

# 2단계: Nginx로 정적 파일 서빙
FROM nginx:stable-alpine
```

```
# Nginx 설정 복사
COPY nginx.conf /etc/nginx/conf.d/default.conf

# 빌드된 정적 파일 복사
COPY --from=builder /app/dist /usr/share/nginx/html

# HTTPS 인증서 디렉토리 마운트를 위해 미리 경로 확보
VOLUME ["/etc/letsencrypt"]

EXPOSE 80
EXPOSE 443
```

```
# nginx.conf

# 1. HTTP 요청 → HTTPS 리디렉션
server {
    listen 80;
    server_name j12a707.p.ssafy.io;

    location /.well-known/acme-challenge/ {
        root /usr/share/nginx/html;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

# 2. HTTPS 요청 처리
server {
    listen 443 ssl;
    server_name j12a707.p.ssafy.io;

    ssl_certificate /etc/letsencrypt/live/j12a707.p.ssafy.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/j12a707.p.ssafy.io/privkey.pem;

    #favicon.ico 서빙
```

```
    location = /favicon.ico {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri /index.html;
    }

    # 정적 파일 서빙
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri /index.html;
    }

    # API 프록시
    location /api/ {
        proxy_pass http://api-gateway:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # API 프록시
    location /internal/ {
        proxy_pass http://api-gateway:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
# Jenkinsfile

pipeline {
  agent any

  environment {
```

```
  IMAGE_NAME = "taemin0323/vanish-fe"
  CONTAINER_NAME = "vanish-fe"
  NETWORK = "vanish-net"
}

stages {
  stage('Checkout') {
    steps {
      checkout scm
    }
  }

  stage('Docker Build & Push') {
    steps {
      // mapbox-token과 imp-code 두 가지 secret 텍스트를 함께 사용
      withCredentials([
        string(credentialsId: 'mapbox-token', variable: 'MAPBOX_TOKEN'),
        string(credentialsId: 'imp-code', variable: 'VITE_IMP_CODE')
      ]) {
        script {
          docker.withRegistry('https://index.docker.io/v1/', 'Docker-Hub') {
            sh """
            docker build -t ${IMAGE_NAME}:latest \\
              --build-arg VITE_MAPBOX_ACCESS_TOKEN=${MAPBOX_TOKEN} \\
              --build-arg VITE_IMP_CODE=${VITE_IMP_CODE} \\
              FE/front
            docker push ${IMAGE_NAME}:latest
            """
          }
        }
      }
    }
  }

  stage('Deploy') {
    steps {
      sh """
      docker rm -f ${CONTAINER_NAME} || true
```

```
        docker pull ${IMAGE_NAME}:latest
        docker run -d --name ${CONTAINER_NAME} \\
          --network ${NETWORK} \\
          -p 80:80 -p 443:443 \\
          -v /root/nginx/cert/live:/etc/letsencrypt/live \\
          -v /root/nginx/cert/archive:/etc/letsencrypt/archive \\
          -v /root/nginx/cert/renewal:/etc/letsencrypt/renewal \\
          ${IMAGE_NAME}:latest
        """
      }
    }
  }
}
```

- Redis

```
docker pull redis

docker volume create redis-volume

# 도커 volume 조회
docker volume inspect redis-volume

docker run -d \
  --name redis \
  --network vanish-net \
  -p 6379:6379 \
  -v redis-volume:/data \
  --restart=always \
  redis
```